

TECHNICAL TEST

Rayhan Kimi Nabiel Athallah

Question 1:

Build a backend service (language/framework of your choice) that lets you record subscriber usage and retrieve what's been recorded. A usage record captures: subscriberId, callMinutes, smsCount, dataUsageMB.

Design the endpoints, HTTP methods, and request/response shapes yourself, and document them in your README. Storage can be in-memory. A simple frontend that consumes this API is not required, but will be viewed as a plus.

Question 1 Solution

Tools used :

- 1.Backend : FastAPI, Pydantic
- 2.Frontend : Plain html

Quick Start :

Requirement : Docker with Compose Plugin.

`docker compose up -d --build` to start.

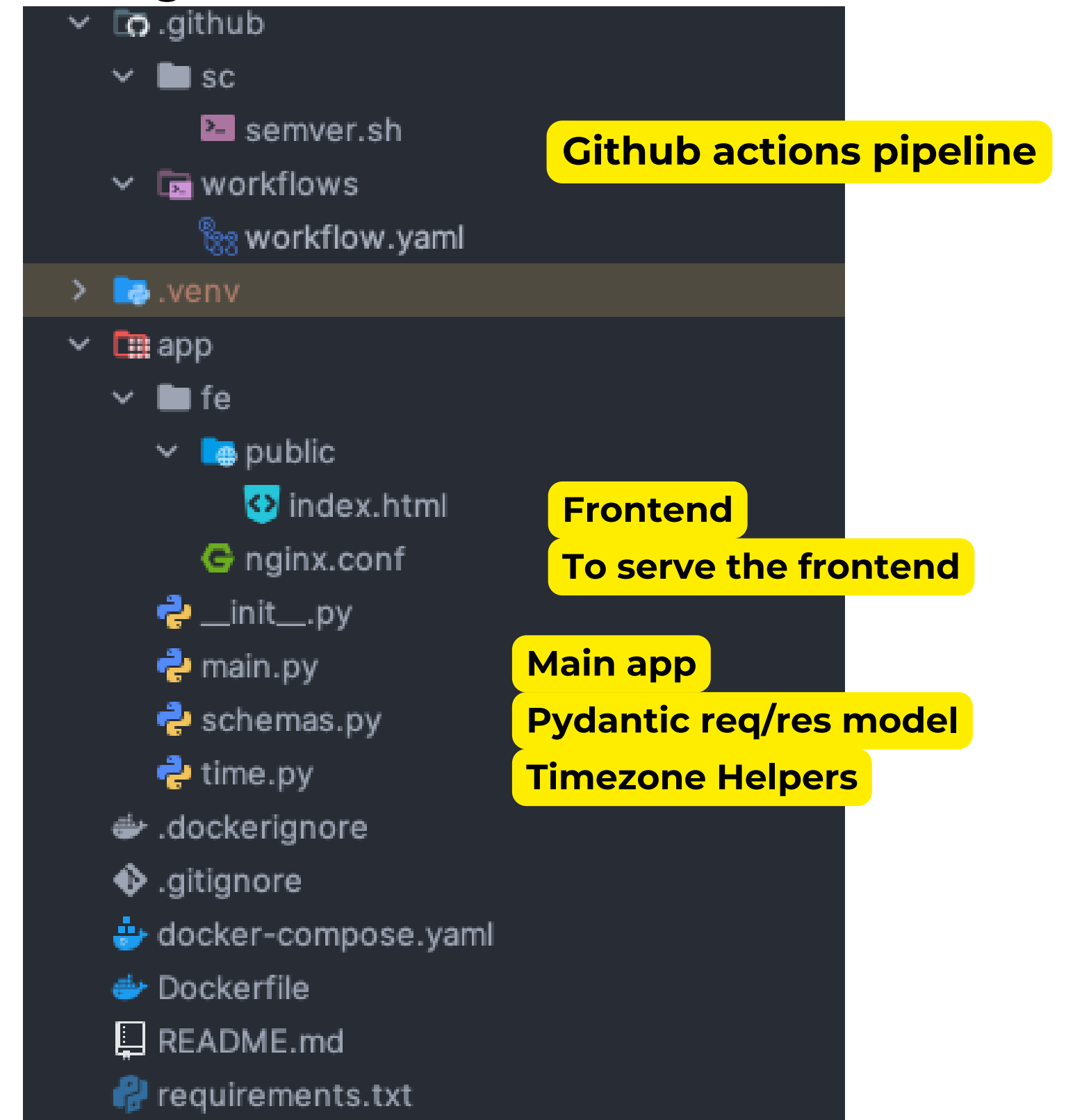
`docker compose down` to stop.

Frontend → <http://localhost:8080>

API → <http://localhost:8000>

Swagger → <http://localhost:8000/docs>

Project Structure :



Question 1 Explanation

Architecture :

Browser → Nginx (:8080) → “/” → static html
→ “/api/*” → api:8000/*

curl → api (:8000)

Data Model :

Field	Type	Set by	Rules
subscriberId	string	client	Required, non empty (after trim whitespace)
callMinutes	integer	client	Required, >= 0
smsCount	integer	client	required, >= 0
dataUsageMB	number	client	required, >= (decimals allowed)
id	integer	server	Unique, auto increment
timestamp	datetime (ISO 8061 UTC)	server	Time the record was stored

Endpoints :

GET /health → Liveness check (used by docker) → Res : 200

POST /usages → Record usage entry → Res : 201

GET /usages → Retrieve recorded usages with filter → Res : 200

Question 1 Explanation

POST /usages (record usage) :

Request Body :

```
1  {
2    "subscriberId": "SUB01",
3    "callMinutes": 40,
4    "smsCount": 10,
5    "dataUsageMB": 1500
6  }
```

Response Body :

```
1  {
2    "subscriberId": "SUB01",
3    "callMinutes": 40,
4    "smsCount": 10,
5    "dataUsageMB": 1500.0,
6    "id": 1,
7    "timestamp": "2026-09-22T19:27:32.648452Z"
8  }
```

Error example :

```
1  {
2    "detail": [
3      {
4        "type": "greater_than_equal",
5        "loc": ["body", "callMinutes"],
6        "msg": "Input should be greater than or equal to 0",
7        "input": -1,
8        "ctx": { "ge": 0 }
9      }
10   ]
11 }
```

GET /usages (retrieve usage) :

Parameter :

- **subscriberId (string)**
- **from (datetime)**
- **to (datetime)**

datetime timezone format :

Format example	Interpreted as :
2026-09-23T08:00:00Z	UTC
2026-09-23T08:00:00+07:00	Given offset (which utc+7)
2026-09-23T08:00 (with no offset)	WIB (UTC+7) by default

Example response (200 ok) :

```
1  [
2    {
3      "subscriberId": "SUB01",
4      "callMinutes": 40,
5      "smsCount": 10,
6      "dataUsageMB": 1500.0,
7      "id": 1,
8      "timestamp": "2026-09-22T19:27:32.648452Z"
9    }
10 ]
```

Question 1 Explanation

Decision in building :

1. **One retrieval with filter instead specialized one (like /subscribers/id/usages).** Filters compose freely, keep API surface small, and give the Q2 snapshot job exactly what it needs (to record for specific time window).
2. **Separate input and output model.** UsageCreate define what client may send and UsageRecord add server owned (which id and timestamp). API contract is therefore explicit in code and in the generated swagger docs.
3. **In memory store + single worker, keep application simple but safe for concurrent request.** ID generation guarded by lock, and with only one worker keeps the data consistent because only using in memory store.

Known Limitation :

1. **Data will be lost when restart & only using one api worker.** Reason : drawback because of memory store. Solution : using persistent storing such as shared database then add more api worker.
2. **No auth.** Reason : to keep application simple. Solution : add API keys or oauth2.
3. **No pagination.** Reason : to keep application simple and because there will be not much data because in memory store. Solution : cursor parameter or limit.

Question 2 :

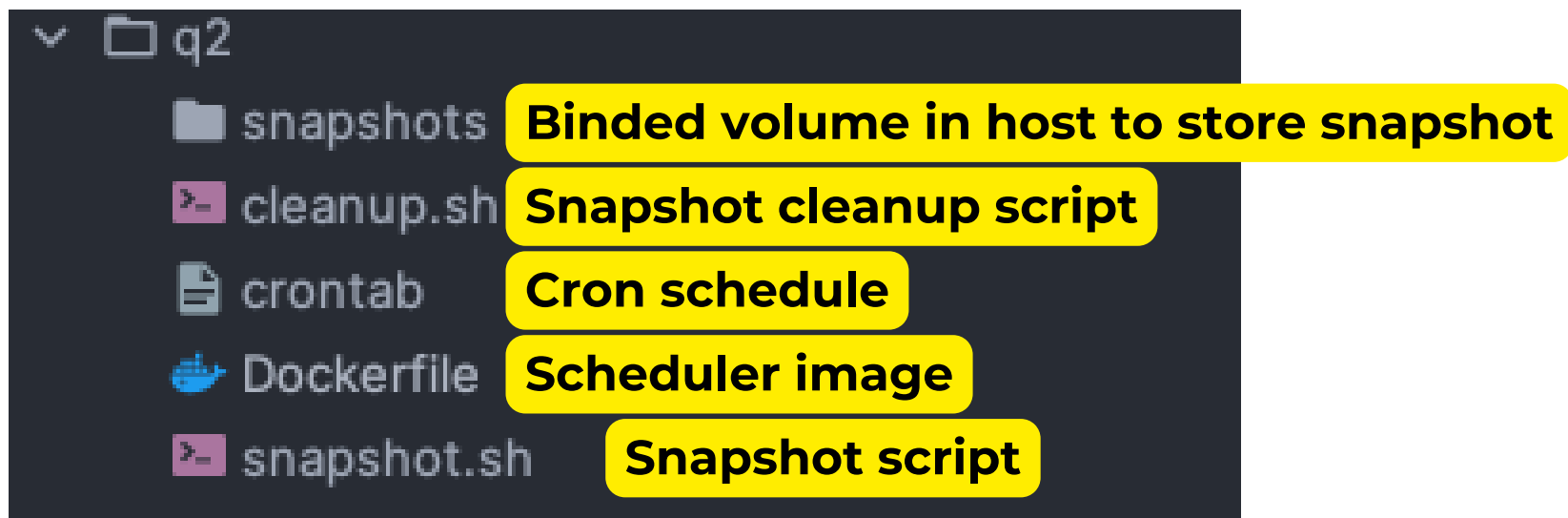
Write a cron job that calls your own retrieval endpoint from Q1 3 times a day (08:00, 12:00, 15:00 WIB) and saves the response as a CSV file. Choose and document your own file naming convention. Write a separate script that removes CSV snapshots older than 30 days.

Question 2 Solution

Tools used :

1. Docker (run alongside other container using compose as scheduler container using Alpine with busybox crond)
2. Other tools : jq, curl, tzdata, bash

Project Structure :



Quick Start :

Requirement : Docker with Compose Plugin.
(Run alongside Q1)

``docker compose up -d --build`` to start.

``docker compose down`` to stop.

Manually Run :

- Take a snapshot for the most recent slot:

`docker compose exec scheduler /app/snapshot.sh`

- Take a snapshot for a specific slot, e.g. to include records you just created:

`docker compose exec -e SNAPSHOT_AT="2026-09-23 12:00" scheduler /app/snapshot.sh`

- Preview the cleanup without deleting anything:

`docker compose exec -e DRY_RUN=1 scheduler /app/cleanup.sh`

- Run the cleanup:

`docker compose exec scheduler /app/cleanup.sh`

- View job output:

`docker compose logs scheduler`

Question 2 Explanation

Architecture :

1. crond run as the container main process and read crontab
2. snapshot.sh will be run 3 times a day with filter that has been built before.
3. Result will be written to /data/snapshots (container) which binded to q2/snapshots on the host.
4. Everyday at 00:30 cleanup script will be executed
5. Job output also goes to stdout, so it shows up in docker compose log.

Model :

1. If the scheduler run at 08:00, time window that will be exported are 15:00 in yesterday and 08:00.
2. If the scheduler run at 12:00, time window that will be exported are in 08:00 and 12:00.
3. If the scheduler run at 15:00, time window that will be exported are in 12:00 and 15:00
4. Exit codes for script : 0 success, 1 API req/csv conversion failed, 2 invalid input caused by malformed variables.

Naming :

File naming convention:

usage_snapshot_YYYYMMDD_HHMM_WIB.csv

Example: **usage_snapshot_20260923_1200_WIB.csv**

- HHMM is the end of the exported window (08:00, 12:00 or 15:00), not the time the script ran.
- The format sorts chronologically, so ls lists snapshots in time order.
- Re-running the same slot overwrites the same file instead of creating a duplicate.
- The cleanup script reads each file's age from its name, not from its modification time.

CSV Format :

- Columns: id, subscriberId, callMinutes, smsCount, dataUsageMB, timestamp, the same fields as a Q1 usage record.
- Timestamps are in WIB with an explicit offset, e.g. 2026-09-23T07:12:05+07:00.
- An empty window still produces a file containing only the header, so "no usage" can be told apart from "the job failed".

Question 2 Explanation

Decision :

1. Container instead host cron so reviewer don't have to configure anything.
2. One window per snapshot to avoid duplications.
3. Write snapshot in tmp before writing, there is never a half written csv. Failure means no file (csv never contains an error message).
4. Windows derived from the schedule (idempotent) re running a slot overwrite the same file.

Known Weakness:

1. **BusyBox crond runs as root.** Supercronic is an alternative to BusyBox but require installing additional binary.
2. **CSV(s) are stored only on local disk.**
Solution : store it also in object storage such as in AWS S3.
3. **On a linux host, CSV(s) are created as root. Regular user needs sudo to delete them.**
4. **File that doesn't match naming convention will never be deleted by cleanup script.**

Question 3 :

Assume the CSV snapshots produced by your Q2 script have been loaded into a usage table with the same fields as your Q1 API's usage records (subscriberId, callMinutes, smsCount, dataUsageMB, timestamp). Write SQL to:

- 1. Insert a new subscriber named Fajar, Basic plan, activated 24 January 2024.**
- 2. Update Fajar's plan to Premium.**
- 3. Calculate the total data usage (sum of dataUsageMB across all snapshots) for all Premium-plan subscribers.**
- 4. Sort and display the top 3 subscribers by total data usage across all snapshots.**
- 5. Write a subquery to find subscribers whose average call minutes per snapshot is less than or equal to 30**

Question 3 Solution

Quick Start :

Image to run sql query already included in docker compose including its schema and seeder query.

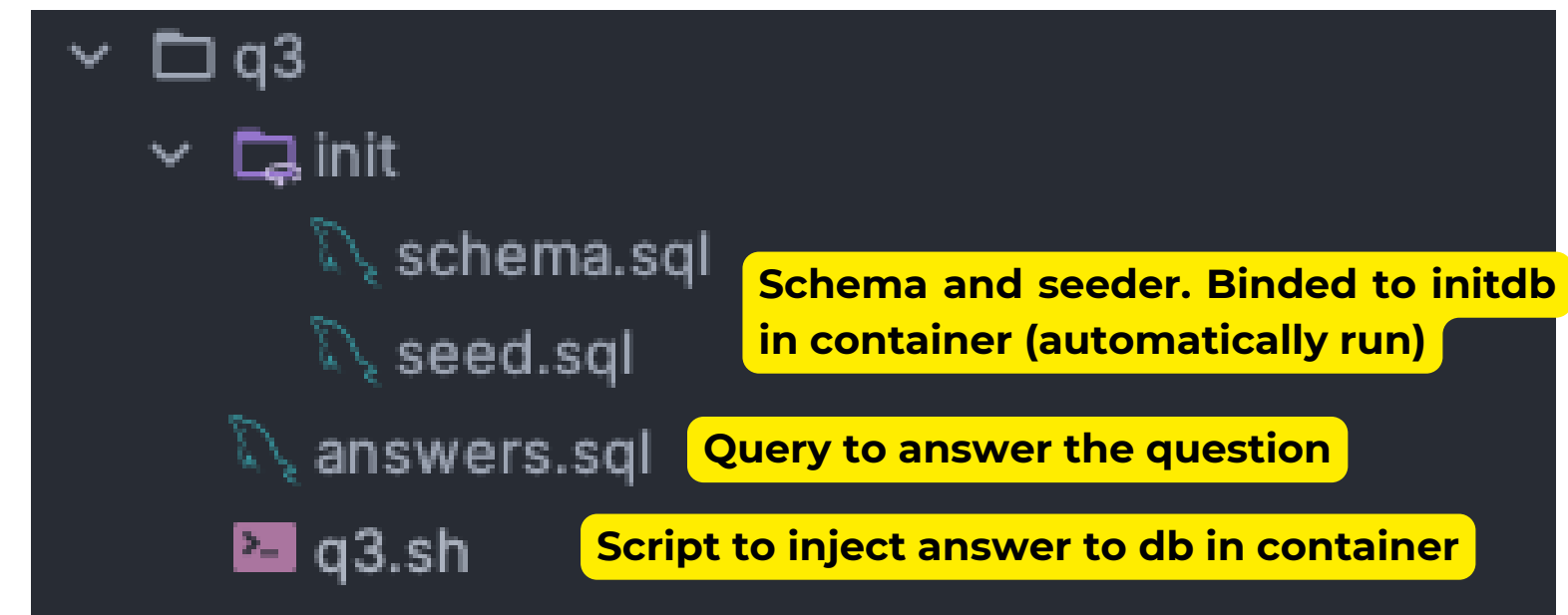
Requirement : Docker with Compose Plugin.
(Run alongside Q1)

`**docker compose up -d --build --wait**` to start.

`**docker compose down**` to stop.

`**docker compose up -d --force-recreate -V db**`
to reset database.

“**q3/q3.sh**” is a script to run answer query after db has been initialized



Question 3 Solution

Result Output :

```
[+] Running 1/1
✓ Container huawei-db-1 Healthy 0.5s
BEGIN

= 1. Insert a new subscriber: Fajar, Basic plan, activated 24 January 2024 =
  id | name | plan | activation_date
-----+-----+-----+-----
SUB07 | Fajar | Basic | 2024-01-24
(1 row)

INSERT 0 1

= 2. Update Fajar's plan to Premium =
  id | name | plan | activation_date
-----+-----+-----+-----
SUB07 | Fajar | Premium | 2024-01-24
(1 row)

UPDATE 1

= 3. Total data usage across all snapshots for Premium-plan subscribers =
total_premium_data_mb
-----
16800.00
(1 row)
```

```
Breakdown per Premium subscriber (LEFT JOIN keeps Fajar, who has no usage yet)
 id | name | total_data_mb
-----+-----+-----
SUB02 | Sari | 11800.00
SUB05 | Rian | 5000.00
SUB07 | Fajar | 0.00
(3 rows)

= 4. Top 3 subscribers by total data usage across all snapshots =
  id | name | plan | total_data_mb
-----+-----+-----+-----
SUB02 | Sari | Premium | 11800.00
SUB04 | Dewi | Family | 9000.00
SUB05 | Rian | Premium | 5000.00
(3 rows)

= 5. Subscribers whose average call minutes per snapshot is ≤ 30 (subquery) =
  id | name | avg_call_minutes
-----+-----+-----
SUB03 | Budi | 20.00
SUB06 | Nia | 25.00
(2 rows)

ROLLBACK
Rolled back: the database is back to the seeded state.
```

Question 4 :

```
function getTotalUsageMB(records) {  
  return records.reduce((total, record) => {  
    total += record.dataUsageMB;  
  });  
}
```

Identify the root cause of the bug, fix it, and explain in writing why it broke and how you would prevent this class of bug in the future.

Question 4 Solution

Root Cause :

- The callback has no return. With braces {}, an arrow function returns undefined unless it returns explicitly. `total += ...` only changes a local variable, and `reduce` uses the callback return value as the next total.
- There is no initial value. Without the second argument, `reduce` starts with the first array element (a record object) instead of 0.

Why its fatal? :

- For two or more records, JavaScript throws no error. The function silently returns undefined, and the problem only surfaces later, for example as NaN or a blank value on a dashboard, far away from its cause.

Fix :

```
1 function getTotalUsageMB(records) {  
1   return records.reduce((total, record) => total + record.dataUsageMB, 0);  
2 }
```

- Arrow function without braces, so that result expression automatically returned.
- Add initial value of 0 so that math operation always start as a number

Prevention :

As a DevOps, the best way to prevent this :

1. **Integrate linter.** ESLint linter will catch precisely this bug. (Callback using `reduce`, `map`, `filter` without returning value).
2. **Typechecking.** Adding interface for such type to avoid type mismatch.
3. **Integrate unit test and adopting TDD.**
4. **Integrate point 1-3 as gate in CI pipeline and run it on every PR check.**

Thank You